L17-20: Quantum Gates, Noise, Qiskit, Quantum Algorithms

Intuition, Mathematics, and Implementation

Vishvendra S. Poonia
Department of Electronics and Communication Engineering
Indian Institute of Technology Roorkee
vishvendra@ece.iitr.ac.in
vishvendra.punia@gmail.com

This session (17-18, 19-20)

Quantum Gates

Noise in Qubits

Qiskit

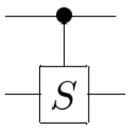
Quantum Algorithms

Quantum Gates

Quantum gates = Unitary operators

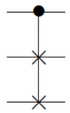
- **Linear**
- **Reversible**

controlled-phase

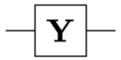
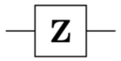
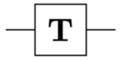
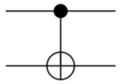
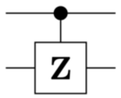
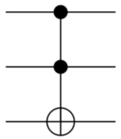


$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & i \end{bmatrix}$$

Fredkin (controlled-swap)



$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Operator	Gate(s)		Matrix
Pauli-X (X)			$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y (Y)			$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z (Z)			$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Hadamard (H)			$\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
Phase (S, P)			$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
$\pi/8$ (T)			$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$
Controlled Not (CNOT, CX)			$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
Controlled Z (CZ)			$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$
SWAP			$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Toffoli (CCNOT, CCX, TOFF)			$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$

Quantum Gates: Universal set of quantum gates

Quantum gates = Unitary operators

- **Linear**
- **Reversible**

- **Classical Universal: {AND, NOT} {NAND}, {NOR}**
- **A gate set G is universal if any unitary operator U can be approximated to arbitrary accuracy using a finite sequence of gates from G.**
- **A gate set becomes universal if it contains:**
  - **Arbitrary Single-Qubit Rotations** i.e. any rotation on the Bloch sphere
  - **At Least One Entangling Two-Qubit Gate:** CNOT, CZ, iSWAP
- **Examples: {H,T,CNOT}, {Rx(θ),Rz(θ),CNOT}, {Toffoli,H}**
- **Geometrically, for single qubits, any point on Bloch sphere should be reachable from any other point.**
- **Similarly, two-qubit universality means that any entangled state can be produced**

Operator	Gate(s)		Matrix
Pauli-X (X)	X	⊕	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y (Y)	Y		$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z (Z)	Z		$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Hadamard (H)	H		$\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
Phase (S, P)	S		$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
$\pi/8$ (T)	T		$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$
Controlled Not (CNOT, CX)	●⊕		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
Controlled Z (CZ)	●Z	●●	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$
SWAP	╳	××	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Toffoli (CCNOT, CCX, TOFF)	●●⊕		$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$

Clifford gates: {H, S, CNOT}

Circuits with only Clifford gates (H, S,CNOT) can be efficiently simulated classically (Gottesman–Knill theorem) —> **In polynomial time**

To achieve universality we must add a non-Clifford gate such as T gate

Quantum Gates

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Quantum gates = Unitary operators i.e. $U^\dagger U = I$

Single Qubit Gates:

- **Pauli Operators: generate rotations.**

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

- **Hadamard Gate:**
  - **It creates superposition:**

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

- For n qubits, the combined action is:

$$H_n = \underbrace{H \otimes H \otimes \ldots \otimes H}_{n \text{ times}}$$

- **The resulting states:** — uniform quantum superposition state

$$H_n|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} |j\rangle$$

$$|x\rangle = |x_1 x_2 \ldots x_n\rangle$$

- For a computational basis state:
- **The generaliz**

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle$$

where is the bitwise inner product mod 2.

$$x \cdot y = x_1 y_1 \oplus x_2 y_2 \oplus \cdots \oplus x_n y_n$$

- **Phase Gate:**

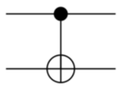
$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

- **General Rotat**

$$R_{\hat{n}}(\theta) = e^{-i\theta \hat{n} \cdot \vec{\sigma}/2}$$

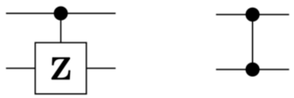
Two-Qubit Gates

Controlled Not (CNOT, CX)



$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Controlled Z (CZ)



$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

SWAP

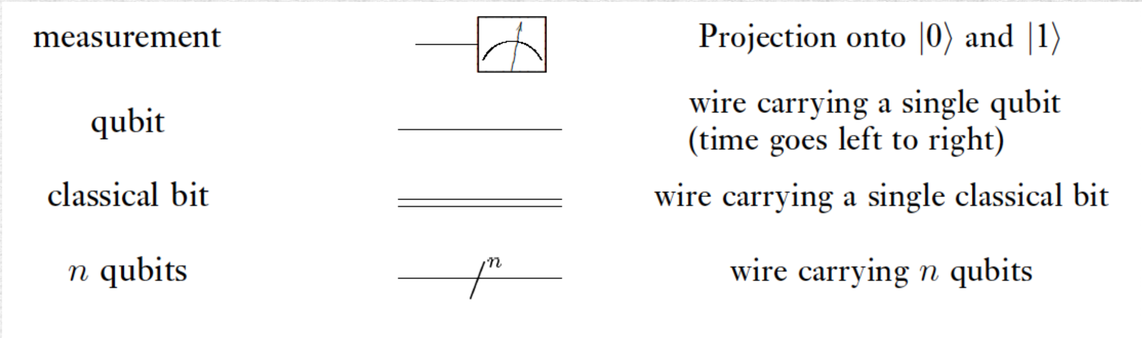
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

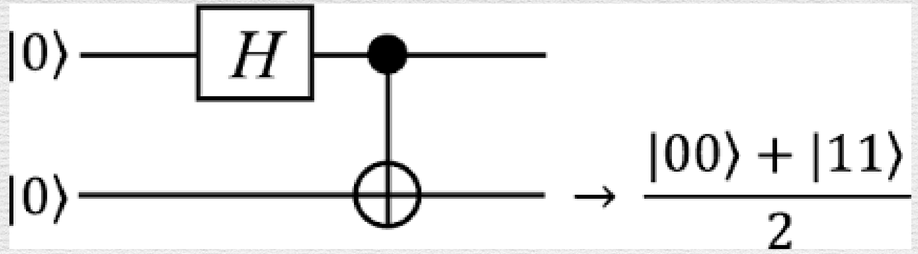
What is a quantum Circuit?

A quantum circuit is a model of quantum computation where information is processed by applying a sequence of unitary operations (quantum gates) on qubits, followed by measurement.

Core Ingredients

- Qubits ($|0\rangle$, $|1\rangle$ superpositions): $|\psi\rangle \in C^{2^n}$
- Quantum Gates (Unitary Operators): $U^\dagger U = I$
- Measurement (Projective): $p(0) = |\langle 0 | \psi\rangle|^2$





A simple quantum circuit with 2-qubit that produces the entangled state

A Brief Introduction to Qiskit: Demo

Introduction to Qiskit

Qiskit is an open-source software framework used to program, simulate, and run quantum algorithms.

Developed by IBM, Qiskit provides the tools needed to move from quantum theory to real quantum hardware.

Why Qiskit?

- **Quantum algorithms are expressed mathematically as:** $|\psi_{out}\rangle = U\,|\psi_{in}\rangle$
- **Qiskit can:**
  - Build quantum circuits
  - **Simulate quantum algorithms on classical computers**
  - **Run QAlgorithms on real quantum processors**
  - **Incorporate noise and error effects**
  - Demonstrate prototype quantum alogrithm applications

Qiskit Conceptual Workflow

Quantum Algorithm

Quantum Circuit

Simulation / Hardware Execution

Measurement Results

Introduction to Qiskit

- **Quantum algorithms are expressed mathematically as:** $|\psi_{out}\rangle = U\,|\psi_{in}\rangle$
- **For this, We must decompose the algorithm into elementary quantum gates.**
- **Step 1: Algorithm → Sequence of Unitary Operations**
  - The algorithm is first written as a sequence of logical gates such as Hadamard H, Pauli gates X , Y , Z, Phase gates and/or Controlled operations
  - $U = U_n U_{n-1} \ldots U_2 U_1$; Each U_i acts on one or two qubits
- **Step 2: Quantum Logic Gates → Quantum Circuit**
  - The sequence of gates is drawn as a quantum circuit.
  - This circuit is written using Qiskit.

Why do we need the Transpiler

- **Real quantum processors cannot execute arbitrary circuits directly.**
- **Each device has physical constraints, such as:**
  - **Limited Gate Set:** The hardware may only support gates like: X, SX, RZ, $CNOT$ but our circuit might contain: H, $SWAP$, Toffoli, Arbitrary rotations etc.
  - **Limited Qubit Connectivity:** Not all qubits can interact.
    - Example connectivity graph:

```
0 – 1 – 2
    |
    3
```

    - But in our circuit, if we have: CNOT(0,3)
    - The hardware cannot connect them directly, it must be rewritten.

Introduction to Qiskit

Why do we need the Transpiler

- **Real quantum processors cannot execute arbitrary circuits directly.**
- **Each device has physical constraints, such as:**
  - **Limited Gate Set**
  - **Limited Qubit Connectivity**

Role of the Transpiler

- **The transpiler converts a logical quantum circuit into a hardware-compatible circuit. Major tasks:**
- **Task 1: Gate Decomposition**
  - **Convert unsupported gates into supported ones.**
  - **E.g.** $H = RZ(\pi) \cdot SX \cdot RZ(\pi)$**. Therefore, Hadamard gate becomes a series of supported gates**
- **Task 2: Qubit Mapping**
  - Maps logical qubits → physical qubits.
  - **E.g.:** Logical qubit 0 → physical qubit 5
  - Logical qubit 1 → physical qubit 7
- **Task 3: Routing (SWAP insertion)**
  - If two qubits are not connected, the transpiler inserts SWAP gates to move them.
  - E.g.: Desired operation: CNOT(q0,q3)
  - But Hardware connectivity: q0—q1—q2—q3
  - The transpiler will insert:
    - SWAP(q0,q1)
    - SWAP(q1,q2)
    - CNOT(q2,q3)
- **Task 4: Circuit Optimization** - The transpiler simplifies circuits by canceling inverse gates merging rotations reducing depth
  - E.g.: CNOT - CNOT ⟹ Identity

The transpiler acts like a compiler, translating a high-level quantum circuit into the low-level instructions that the backend (simulator or hardware) can execute efficiently.

Complete Qiskit Workflow

- **Quantum algorithm = mathematical idea**
- **Quantum circuit = sequence of quantum gates implementing the idea**
- **Transpiler = compiler that converts the circuit into instructions the simulator/hardware can actually run.**
- **Why do we need transpiler even with the simulator:**
  - Even though a simulator runs on a classical computer and is not constrained by physical hardware, the circuit must still be compiled into an executable representation
  - **Step 1:** It first converts Abstract Circuits into Backend Instructions
    - E.g. when we write: qc.h(0); qc.cx(0,1) => this is a high-level description.
    - The transpiler converts the circuit into this **backend-compatible instruction set**.
  - **Step 2: Standardize the Gate Set:** Quantum circuits may contain gates such as H, T, SWAP, Toffoli, arbitrary rotations
    - The simulator internally prefers a standard basis gate set such as {RZ,SX,X,CNOT}. The transpiler decomposes gates accordingly.
  - **Step 3: Circuit Optimization:** The transpiler simplifies circuits before execution, e..g X-X gates are converted into Identity.
    - This reduces: Circuit depth, number of operations, simulation time
  - **Step 4: Efficient Simulation Scheduling:** The simulator executes circuits using optimized numerical routines.

Quantum Algorithm

Logical Quantum Circuit

Transpiler

Hardware-Compatible Circuit

Execution (Simulator or Quantum Processor)

Measurement Results

Consistency with Real Hardware Execution: One of the design goals of Qiskit is that **the same code should run on both simulators and real devices**. Circuit → Transpiler → Backend

Whether the backend is a simulator like Qiskit Aer or a real quantum processor. This ensures that the simulation accurately reflects how the circuit would run on hardware.

No-Cloning Theorem

It is impossible to create an exact copy of an arbitrary unknown quantum state.

Statement: There is no unitary operator U such that: $U(|\psi\rangle|0\rangle) = |\psi\rangle|\psi\rangle$ for all states $|\psi\rangle$.

Proof: Let's assume cloning works, i.e.

$U(|0\rangle|0\rangle) = |0\rangle|0\rangle$

$U(|1\rangle|0\rangle) = |1\rangle|1\rangle$

Let's consider: $|\psi\rangle = 1/\sqrt{2}(|0\rangle + |1\rangle)$

By Linearity: $U(|\psi\rangle|0\rangle) = 1/\sqrt{2}(|00\rangle + |11\rangle)$

But Cloning requires:

$|\psi\rangle|\psi\rangle = 1/\sqrt{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$

These are NOT equal.

That's a contradiction.

Proof: Let's say a universal cloning machine exists such that:

$|\psi\rangle|0\rangle \rightarrow |\psi\rangle|\psi\rangle$

Let's assume, it works for two states $|\psi\rangle, |\phi\rangle$

By linearity of quantum mechanics: $|\psi\rangle|0\rangle \rightarrow |\psi\rangle|\psi\rangle$

$|\phi\rangle|0\rangle \rightarrow |\phi\rangle|\phi\rangle$

Now, let's consider the superposition: $1/\sqrt{2}((|\psi\rangle + |\phi\rangle))$

Linearity requires: $1/\sqrt{2}(|\psi\rangle + |\phi\rangle)|0\rangle \rightarrow 1/\sqrt{2}((|\psi\rangle|\psi\rangle + |\phi\rangle|\phi\rangle)$

But Cloning requires:

$1/\sqrt{2}(|\psi\rangle + |\phi\rangle)1/\sqrt{2}(|\psi\rangle + |\phi\rangle) \rightarrow 1/2(|\psi\rangle|\psi\rangle + |\psi\rangle|\phi\rangle + |\phi\rangle|\psi\rangle + |\phi\rangle|\phi\rangle$

These are NOT equal. Therefore, universal cloning violates linearity. Hence cloning is impossible.

Importance:

- **Quantum cryptography security**
- **No perfect backup of quantum data**
- **Prevents superluminal communication**

Bell's Inequality

The Question:

Can the world be explained by:

- **Locality → no faster-than-light influence**
- **Realism → properties exist before measurement**

Bell (1964): If both assumptions are true, then correlations between distant particles must satisfy a mathematical limit (Bell's Inequality).

But, Quantum mechanics predicts stronger correlations than this limit allows.

Why Quantum Mechanics is Truly Non-Classical

Real experiments show:

- **Bell's inequality is violated**
- **Local realism cannot fully describe nature**

This Means:

- **Entangled particles behave like a single non-local system**
- **Measurement here influences correlations there (not usable for communication)**
- **Reality is more subtle than classical intuition**

Important:

- **Nature is either non-local, non-realistic, or both**
- **And quantum mechanics is valid.**

Quantum Algorithms: Essential Primitives

- **Qubits and State Representation:** $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$
  - This is important because all algorithms manipulate amplitudes.
- **Measurement and Probabilities:** $P(0) = |\alpha|^2, P(1) = |\beta|^2$
  - **Measurement collapses the state.**
  - That's why algorithms must amplify correct answers Why we cannot simply read all amplitudes
- **Unitary Evolution:** $|\psi'\rangle = U|\psi\rangle$
  - It is Reversible, Norm preserving, Implemented as quantum gates
  - Quantum circuits are sequences of unitary operators.
- **Quantum Gates**
  - Superposition (Hadamard Gate): $H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$
  - Entanglement (CNOT)
- **Quantum Parallelism:** $H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$
  - Now if we apply: $U_f|x\rangle|0\rangle = |x\rangle|f(x)\rangle$
  - Quantum computer evaluates the function on all inputs simultaneously.
  - But measurement destroys the parallelism.

- **Quantum Oracles: Almost all quantum algorithms use a black-box function.**
  - Oracle: $U_f|x, y\rangle = |x, y \oplus f(x)\rangle$
  - This is essential for Deutsch–Jozsa, Bernstein–Vazirani, Simon, Grover algorithms
- **Phase kickback: If** $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2} \Longrightarrow U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle$
  - The function value becomes a phase.
  - **This is behind Deutsch-Jozsa, Bernstein-Vazirani, Simon, and Shor algorithms**
- **Amplitude Amplification: Especially relevant for Grover's algorithm**
  - Reflection operators: $R_s = 2|s\rangle\langle s| - I$
  - Rotation in two-dimensional subspace

Quantum Algorithms: Essential Primitives

- The “Quantum” algorithms:
  1. Deutsch–Jozsa
  2. Bernstein–Vazirani
  3. Simon’s Algorithm
  4. Grover’s Algorithm
  5. Shor’s Algorithm

The Deutsch–Jozsa Algorithm

It was the first clean proof that a quantum computer can be provably faster than any classical one.

The problem Deutsch–Jozsa solves:

Given a black-box function (oracle) $f : \{0,1\}^n \to \{0,1\}$

The Deutsch–Jozsa Algorithm

It was the first clean proof that a quantum computer can be provably faster than any classical one.

The problem Deutsch–Jozsa solves: It determins whether a given, hidden Boolean function f(x)

- Is **Constant** (returns the same value, 0 or 1, for all inputs) OR
- **Balanced** (returns 0 for half the inputs and 1 for the other half)

It guarantees this with 100% certainty in a single evaluation (query) of the function.

Given a black-box function (oracle): $f: \{0,1\}^n \rightarrow \{0,1\}$

Classical Requirement: A deterministic classical computer might require up to $2^{n-1} + 1$ queries to distinguish between a constant and balanced function.

Quantum Speedup: The algorithm uses quantum parallelism and interference to achieve an exponential speedup, requiring only one query regardless of the input size n

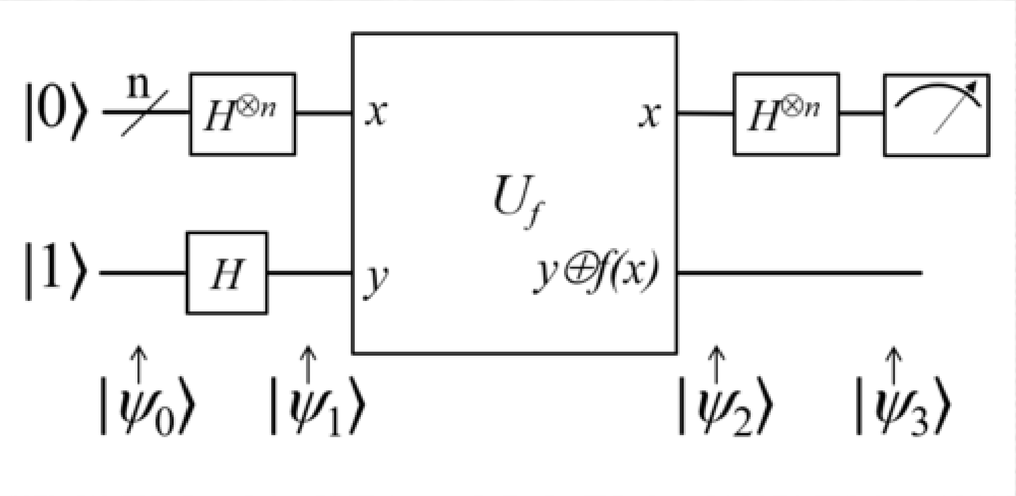
The Deutsch–Jozsa Algorithm

The problem Deutsch–Jozsa solves: It determins whether a given, hidden Boolean function f(x)

- Is **Constant** (returns the same value, 0 or 1, for all inputs) OR
- **Balanced** (returns 0 for half the inputs and 1 for the other half)

It guarantees this with 100% certainty in a single evaluation (query) of the function.

Given a black-box function (oracle): $f: \{0,1\}^n \to \{0,1\}$



Quantum Oracle (given)

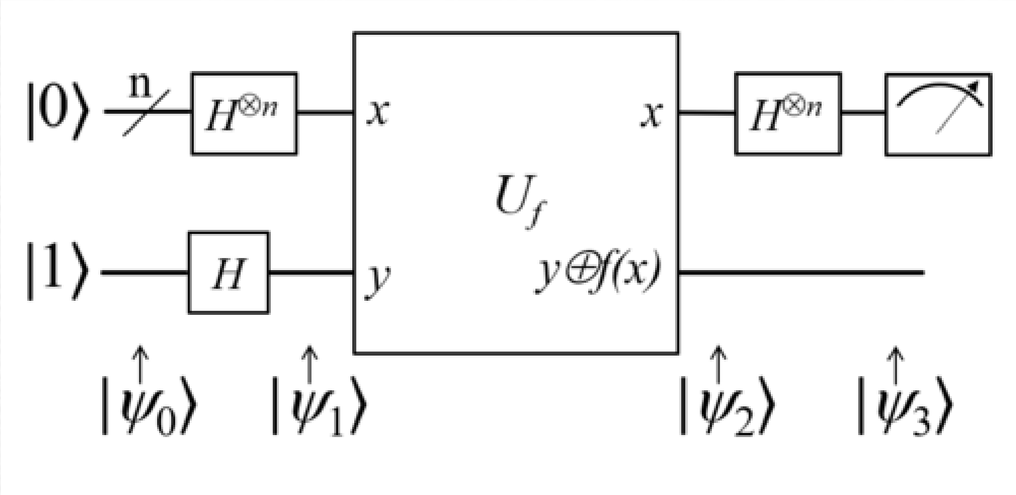
- The oracle acts as:

$$U_f \left|x\right\rangle \left|y\right\rangle = \left|x\right\rangle \left|y \oplus f(x)\right\rangle$$

Key ideas:

- **Quantum parallelism:** Evaluate $f(x)$ for all inputs at once
- **Phase kickback:** Store function values as phases
- **Interference:** Balanced vs constant cause opposite interference patterns

The Deutsch–Jozsa Algorithm: Step-by-step



1. **Initialize the registers:**
   - **Input register: n qubits** $\rightarrow |0\rangle^{\otimes n}$
   - **Output qubit:** $|1\rangle$

$$|\psi_0\rangle = |0\rangle^{\otimes n} |1\rangle$$

2. **Apply Hadamard to all Qubits**

$$H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$$

$$H |1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

5. **Apply the oracle (U_f)**

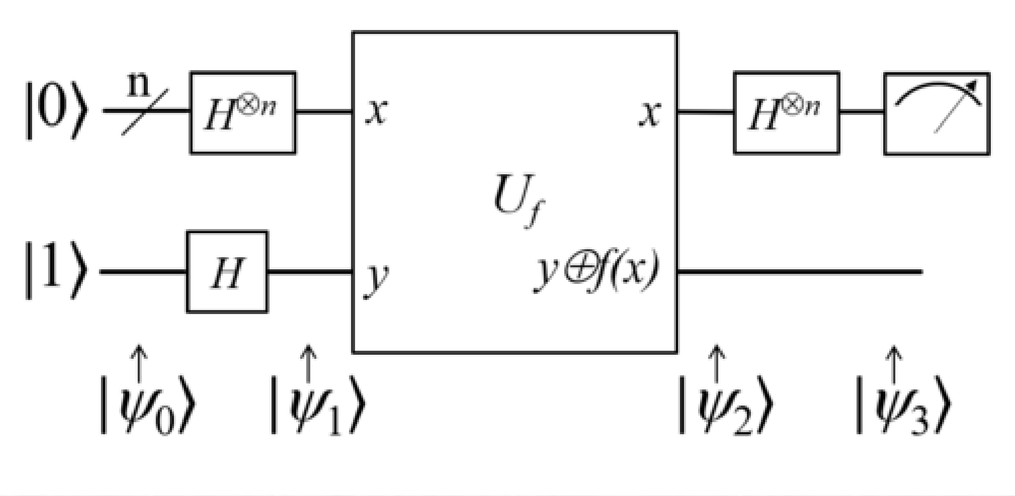
$$|x\rangle (|0\rangle - |1\rangle) \xrightarrow{U_f} (-1)^{f(x)} |x\rangle (|0\rangle - |1\rangle)$$

$$|\psi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_x (-1)^{f(x)} |x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

7. **Apply Hadamards again to the input register:**

$$A(0) = \frac{1}{2^n} \sum_x (-1)^{f(x)}$$

The Deutsch–Jozsa Algorithm: Case Analysis



Case analysis:

- **Constant Function**
  - If $f(x) = 0$: all phases = +1
  - If $f(x) = 1$: all phases = -1
  - Both of them implies: $\sum_x (-1)^{f(x)} = \pm 2^n$
  - Therefore, the final state is: $|0\rangle^{\otimes n}$ with probability 1

- **Balanced Function:** Exactly half + 1, half – 1 → $\sum_x (-1)^{f(x)} = 0$
  - Probability of measuring 0 in the input register is zero. We should observe something else.

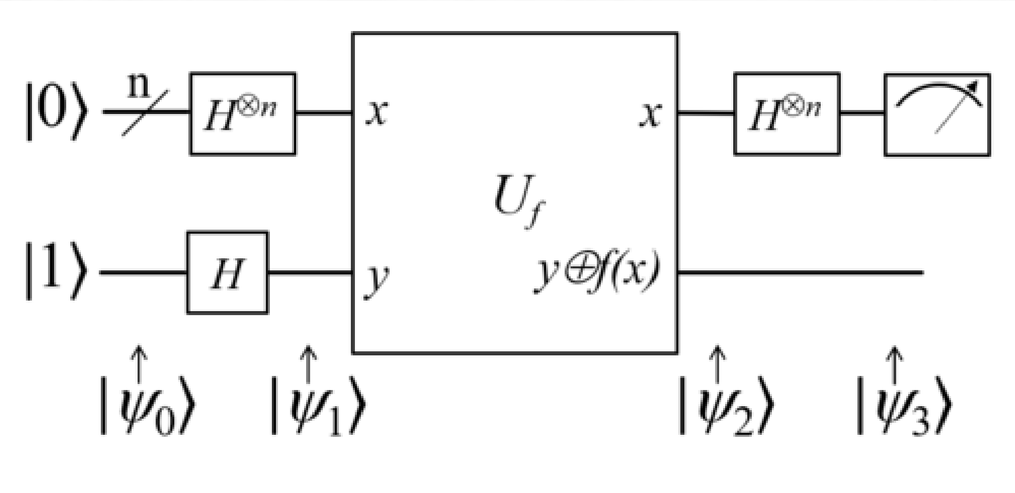
The Deutsch–Jozsa Algorithm

The problem Deutsch–Jozsa solves: It determins whether a given, hidden Boolean function f(x)

$$U_f \left|x\right\rangle \left|y\right\rangle = \left|x\right\rangle \left|y \oplus f(x)\right\rangle$$

- Is **Constant** (returns the same value, 0 or 1, for all inputs) OR
- **Balanced** (returns 0 for half the inputs and 1 for the other half)

It guarantees this with 100% certainty in a single evaluation (query) of the function.



Given a black-box function (oracle): $f: \{0,1\}^n \rightarrow \{0,1\}$

Oracle model (crucial)

- The oracle acts as:

Key ideas:

- **Quantum parallelism:** Evaluate $f(x)$ for all inputs at once
- **Phase kickback:** Store function values as phases
- **Interference:** Balanced vs constant cause opposite interference patterns

Thank you